

Site de reservation

Airlines Reservation

Dossier de rattrapage - preparation au jury

Ce document reprend l'ensemble des corrections et evolutions apportees au projet suite aux retours du premier jury, organisees selon les 3 blocs de competences evalues :

Bloc 1 (Front-End), Bloc 2 (Back-End) et Bloc 3 (Framework).

Stack technique : Flask (Python) + PostgreSQL + HTML/CSS/JS natif

Sommaire

- Retour sur le premier passage : scores et points bloquants
- Bloc 1 : Développement Front-End (HTML/CSS/JS)
- Bloc 2 : Développement Back-End (BDD/PHP-Python/RGPD/roles)
- Bloc 3 : Framework (Flask)
- Ce qui reste à traiter avant le jury

Retour sur le premier passage

Le premier passage a été évalué sur 3 blocs, avec un point commun sur les 3 grilles : le projet fonctionnait mais manquait de profondeur sur des points précis, identifiés et corrigés depuis.

Scores obtenus

- Bloc 1 - Front-End : 37/100 - point bloquant : responsive absent (0/3), aucune librairie JS externe (0/3)
- Bloc 2 - Back-End : 30/100 - point bloquant : un seul projet livré (avec framework) alors qu'un projet en code natif était attendu ; aucun rôle utilisateur ; pas d'UPDATE/DELETE ; RGPD absent (0/3)
- Bloc 3 - Framework : 67/100 - point fort, mais espace d'administration et UPDATE/DELETE demandés en plus

Les corrections présentées ci-dessous répondent directement, point par point, aux critères non-acquis ou partiellement acquis des 3 grilles.

Bloc 1 : Developpement Front-End

C1.a - Integration HTML/CSS des maquettes

Avant 1/3 - integration partielle

- Uniformisation du design des cartes hotel/vol (classes .h-card / .v-card) sur toutes les pages : accueil, liste hotels, liste vols, resultats de recherche, page de reservation - un seul systeme de carte reutilise partout au lieu de 3 designs differents.
- Coherence visuelle complete du parcours utilisateur (header, navigation, footer identiques partout).

C1.b - Responsive (mobile / tablette / desktop)

Avant 0/3 - non acquis

CORRIGE - test explicite sur 1920px / 768px / 375px avec captures

- Formulaire de recherche : refonte complete (debordement horizontal corrige, flex-wrap, champs empiles proprement sur mobile).
- Barre de navigation mobile : passage en mode compact (Accueil/Vols/Hotels visibles, bouton 'Afficher plus' pour le reste des liens) plutot que 9 liens qui debordaient.
- Formulaire de recherche mobile : bouton 'Afficher les options de recherche' qui deroule le formulaire complet, pour ne pas surcharger l'ecran d'entree.
- Correction d'un bug flexbox global (body > main avec flex-shrink:0) qui provoquait un debordement/troncature de toute une page des qu'un element large (tableau RGPD) etait present.
- Menu compte utilisateur : passage en menu deroulant (icone + fleche), avec disposition verticale (icone au-dessus du texte) specifiquement sur mobile.
- Boutons d'action (icones editer/supprimer du dashboard admin) : correction d'un bug de rendu d'icones du a l'absence de reset CSS (appearance: none) sur les boutons natifs.
- Images : integration d'une image dediee au format mobile via la balise <picture> (chargement conditionnel selon la largeur d'ecran, poids reduit de 1.8 Mo a 62 Ko).
- Dashboard admin, page profil, pages legales (mentions/confidentialite/CGV), mot de passe oublie : toutes testees et validees en 375px / 768px / desktop.

C1.c - Accessibilite (RGAA)

Avant 1/3 - partiel

- Correction du curseur 'main' qui s'affichait sur toute la carte hotel/vol alors que seul le bouton Reserver est cliquable (source de confusion pour l'utilisateur).
- Correction d'un pattern casse sur toutes les pages : le bouton de connexion etait un <button> contenant un <a> mal ferme (imbrication HTML invalide, navigation clavier/lecteur d'ecran cassee) - remplace par un <a> unique et valide partout.
- Labels de formulaire associes a leurs champs (attribut for/id) sur les nouveaux formulaires

(profil, mot de passe oublie, administration).

A FAIRE avant jury : police dediee aux utilisateurs dyslexiques (Opendys) non integree.

C1.d - Organisation du CSS

Avant 1/3 - partiel

- Nommage coherent en BEM-like pour les nouveaux composants : .h-card__body, .v-card__footer, .admin-sidebar__nav, .payment-card--danger, etc.
- Feuille de style organisee par sections commentees (PAGE ADMIN, DASHBOARD, PAGES LEGALES...).

C1.e - SEO

Avant 1/3 - partiel

- Correction d'un bug reel identifie en cherchant la cause d'un titre de carte vide : la variable vol.nom n'existait pas en base (colonne inexistante), remplacee par vol.ville - corrige a la fois sur l'accueil et la liste des vols.
- Titres de cartes remplaces par de vraies balises semantiques (h2/h3) au lieu de <div>.

C2.a a C2.c - JavaScript (DOM, validation, requetes asynchrones)

Avant 2/3 sur les 3 - deja acquis, renforce

- Systeme de notifications (toast) generique reutilise pour : succes formulaire contact, rate-limit depasse, erreurs de connexion/inscription, reinitialisation de mot de passe, actions du dashboard admin (creation/modification/suppression de compte).
- Requetes asynchrones fetch() en PUT, PATCH et DELETE (pas seulement GET/POST) : mise a jour du profil, changement de mot de passe, gestion des comptes admin - avec jeton CSRF transmis en en-tete HTTP (X-CSRFToken) sur chaque requete.
- Menus deroulants et bandeaux interactifs geres en JS pur (menu compte, navigation mobile, options de recherche, bandeau cookies RGPD).

C2.d - Librairies JavaScript externes

Avant 0/3 - non acquis

A FAIRE avant jury : aucune librairie JS externe fonctionnelle integree et justifiee (Font Awesome ne compte que comme feuille d'icomes, pas comme librairie applicative).

Bloc 2 : Developpement Back-End

Point bloquant principal du jury

"Un seul projet est presente et celui-ci utilise un framework, alors qu'un projet realise en code natif/pur etait attendu." Ce point est traite en priorite absolue : un second backend, sans aucun framework (uniquement http.server + psycpg2), est en cours de construction en parallele de ce document, sur les memes donnees PostgreSQL.

C3.a - Modele de donnees

Avant 1/3 - partiel

- Schema relationnel complet : compte_client, hotels, vols, destinations, reservations_hotel, reservations_vol, paiements, formulaire_contact - avec clef etrangeres entre reservations et hotels/vols/comptes.
- Ajout d'une colonne is_admin (role utilisateur) et de colonnes de reinitialisation de mot de passe (reset_token_hash, reset_token_expires) sur compte_client.

A ameliorer avant jury : le critere demande l'exploitation de donnees externes via une API - non encore integre (ex : donnees meteo ou taux de change pour les destinations).

C3.b - Construction de la base de donnees

Avant 2/3 - acquis

- Contraintes NOT NULL, UNIQUE (email), FOREIGN KEY entre toutes les tables liees, types de champs coherents (BOOLEAN pour is_admin, TIMESTAMP pour les expirations de token).

C3.c - Requetes SQL (CRUD complet)

Avant 0/3 - non acquis

CORRIGE - CRUD complet demontre et teste sur plusieurs ressources

- SELECT (lister), INSERT (creer), UPDATE (modifier), DELETE (supprimer) implementes et testes sur les comptes utilisateurs, exposes via de vraies routes HTTP : GET, POST, PUT, PATCH et DELETE (verbe distinct de PUT, utilise volontairement sur le changement de mot de passe pour demontrer la difference : PUT = remplacement complet, PATCH = modification partielle).
- Jointures SQL (JOIN) utilisees pour rattacher les reservations a leurs hotels/vols/destinations, recalcul systematique du prix cote serveur (jamais fait confiance a une valeur envoyee par le formulaire).
- Suppression en cascade geree manuellement (les reservations liees a un compte sont supprimees avant le compte lui-meme, en l'absence de ON DELETE CASCADE en base).

C3.d - RGPD

Avant 0/3 - non acquis

CORRIGE - mise en conformite complete

- 3 pages legales creees : mentions legales, politique de confidentialite (tableau des donnees collectees par formulaire, finalite, base legale), CGV.
- Bandeau cookies RGPD (Accepter/Refuser) sur toutes les pages, avec lien vers la politique de confidentialite.
- Droits utilisateur concrets et fonctionnels depuis la page profil : consultation, modification (nom/prenom/email), et suppression definitive du compte (avec confirmation par mot de passe).
- Donnees sensibles protegees : mots de passe haches (bcrypt, jamais en clair), tokens de reinitialisation de mot de passe stockes uniquement sous forme d'empreinte SHA-256 (jamais en clair, meme principe que pour un mot de passe).

C4.a a C4.d - Conception, langage, POO, architecture MVC

Avant 1/3 a 2/3 - partiel a acquis

- Gestion d'erreurs systematique (try/except/finally) sur chaque route, jamais d'exposition de stack trace ou de requete SQL au client.
- Separation claire route (controleur) / template (vue) / requetes SQL (modele) deja en place, renforcee sur les nouvelles fonctionnalites (profil, administration, mot de passe oublie).

A ameliorer avant jury : pas de veritable systeme de classes avec heritage/namespaces (le projet reste principalement procedural, hors classe User).

C4.e - Roles utilisateur et securite des acces

Avant 0/3 - non acquis

CORRIGE - systeme de roles complet, teste en conditions d'attaque reelles

- Ajout d'un role administrateur (is_admin), distinct des comptes clients, avec un decorateur admin_required qui bloque a 403 tout acces non autorise (different d'un simple login_required : un utilisateur peut etre connecte mais non autorise).
- Dashboard d'administration separe (sidebar dediee, sans le header/footer du site client), avec CRUD complet sur les comptes clients (creer, lister, modifier, supprimer).
- Tests de securite reels effectues avec 2 comptes distincts (client + admin) : un compte client authentifie, meme avec un jeton CSRF valide, ne peut ni creer de compte admin, ni s'auto-promouvoir admin, ni supprimer un autre compte - verifie a chaque fois par une requete HTTP reelle, pas juste une relecture de code.
- Protection anti-verrouillage : un administrateur ne peut pas se retirer ses propres droits ni se supprimer lui-meme depuis le dashboard (evite un blocage accidentel).
- Reinitialisation de mot de passe securisee : jeton aleatoire (32 octets), usage unique, expiration a 30 minutes, envoi par email reel (SMTP), message identique que l'email existe ou non en base (anti-enumeration de comptes, meme principe applique au message d'erreur de connexion).

C4.f et C4.g - Travail en equipe et livraison testee

Avant 1/3 - partiel

- Chaque fonctionnalité livrée a été testée par des requêtes HTTP réelles (via un client de test et curl), pas uniquement par relecture de code : création/suppression de compte, injection SQL, CSRF manquant, IDOR, mots de passe faibles, tokens expirés - tous vérifiés avec preuve à l'appui (code de statut HTTP, contenu de la réponse, état réel de la base de données).

Bloc 3 : Framework (Flask)

"Le framework est compris et bien pris en main. Le projet est fonctionnel, mais pourrait être davantage approfondi, notamment avec la mise en place d'un espace d'administration et de fonctionnalités complètes de gestion des données : ajout, modification (UPDATE) et suppression (DELETE)."

Reponse directe au commentaire du jury

CORRIGE

- Espace d'administration complet et fonctionnel : dashboard avec sa propre mise en page (sidebar), liste de tous les comptes clients, création, modification et suppression - les 3 actions explicitement demandées par le jury.
- Redirection automatique vers le dashboard dès la connexion si le compte est administrateur, sans toucher au parcours de connexion des clients.
- Toutes les routes du dashboard sont protégées (403 pour tout compte non-admin, redirection login pour tout visiteur non connecté) et testées dans les deux cas.

Points déjà bien notés, renforcés depuis

- Sessions Flask-Login, protection CSRF (Flask-WTF), rate limiting (Flask-Limiter) sur toutes les routes sensibles (connexion, inscription, contact, mot de passe oublié).
- Gestion des erreurs HTTP (429 trop de requêtes, 403 accès refusé, 404 ressource introuvable) avec des pages/messages dédiés plutôt que des erreurs brutes.

Ce qui reste a traiter avant le jury

Liste honnete des points identifiés dans les grilles d'evaluation et pas encore corriges, par ordre de priorite.

Priorite 1 - bloquant Bloc 2

- Backend sans framework (http.server pur) : routeur maison, sessions signees manuellement, CSRF manuel, templating par remplacement de chaine avec echappement HTML manuel - sur un perimetre reduit mais complet (GET/POST/PUT/DELETE, roles, RGPD) plutot qu'un clone integral du site Flask.

Priorite 2 - gains rapides

- Police Opendys pour les utilisateurs dyslexiques (C1.c).
- Integration justifiee d'une vraie librairie JS externe, par exemple un carrousel d'images sur la page d'accueil (C2.d).
- Exploitation d'une donnee externe via une API publique, par exemple la meteo ou le taux de change pour chaque destination (C3.a).

Priorite 3 - si le temps le permet

- Un debut de structuration en classes/POO cote back-end (au-dela de la seule classe User).
- Tests unitaires automatises (pytest) en complement des tests manuels deja effectues.

Document de preparation au jury de rattrapage - Airlines Reservation (Flask + PostgreSQL).